

TAREA 2

Grupo 6 - Sistema TicketBeat
Diseño de Software

Integrantes:

Pablo Ojeda
Daniel Negrete
David Matos
Fernando Zambrano

JUSTIFICACIÓN

PÁRRAFO 1 — Patrón Factory Method

TicketBeat maneja múltiples tipos de boletos (asientos reservados, entradas generales, pases VIP), cada uno con lógica de negocio, accesos y precio base distintos. Se define una clase abstracta CreadorBoleto con un método de fabricación (crearBoleto()), delegando la instanciación a subclases concretas (CreadorBoletoVIP, CreadorBoletoGeneral, CreadorBoletoAsientoReservado), cada una encapsulando la inicialización de su variante. Este patrón aísla la instanciación cumpliendo el principio Abierto/Cerrado: agregar un nuevo tipo (ej. "Meet & Greet") solo requiere una nueva subclase, sin modificar el flujo de compra ni usar if/else. Factory Method es adecuado porque existe una única jerarquía de producto (Boleto) con variantes, no familias de objetos relacionados que deban coordinarse entre sí.

PÁRRAFO 2 — Patrón Strategy

En el párrafo 2 dice que luego que el usuario reserve su entrada puede pagarla. Para realizar esta acción el usuario tiene distintas vías de pago, como tarjetas de crédito, transferencias electrónicas y servicios de pago móviles. Cada vía de pago exige que se cumplan ciertos pasos para completarla, estos pasos no tienen porque ser los mismos, ni estar en el mismo orden. Entonces el sistema debe ser capaz de elegir en tiempo de ejecución que estrategia o secuencia de pasos realizar para cancelar la entrada.

PÁRRAFO 3 — Patrón Decorator

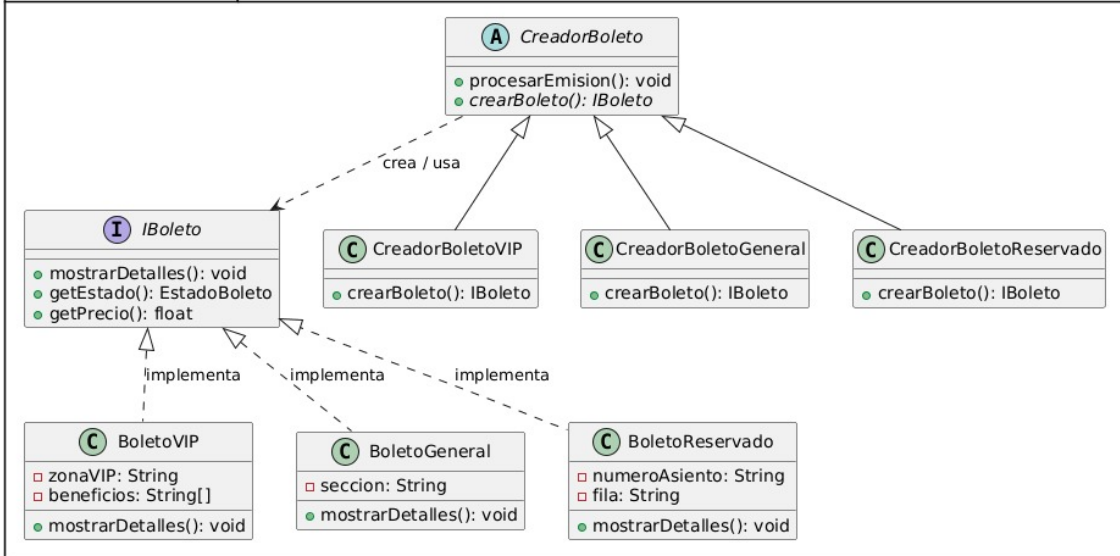
Para agregar restricciones a los eventos ya creados, sin tener que eliminarlos y volviéndolos a crear con las nuevas restricciones. Lo mejor es el patrón decorator ya que nos permite envolver el evento con una reestricción más

PÁRRAFO 4 — Patrón Chain of Responsibility

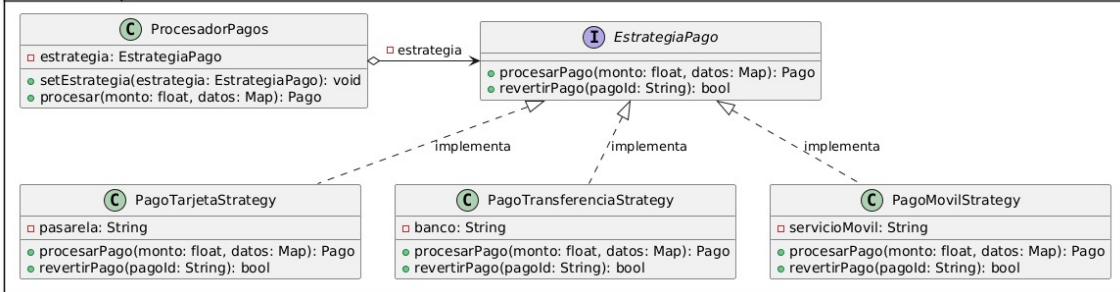
El caso presenta una cadena de manejadores Soporte -> Administración, donde: el cliente (usuario) genera una solicitud (incidente) sin conocer qué manejador la resolverá finalmente (desacoplamiento cliente-receptor); cada manejador (agente de soporte, luego administración) recibe la solicitud y decide autónomamente si la resuelve o la reenvía al siguiente eslabón; y la solicitud se propaga secuencialmente hasta encontrar un manejador que la resuelva (administración, en última instancia).

DIAGRAMAS DE CLASES

FactoryMethodBoletos



StrategyPago



DecoratorPoliticas

